

Memoria Técnica de Entrega — Ejercicios React

Este documento contiene la memoria técnica oficial, la justificación de los resultados de aprendizaje (RA) y los enlaces de entrega para la práctica de React.

 Datos de Entrega Oficiales

Entregable	Enlace de Acceso	Estado
Código Fuente (GitHub)	github.com/manolorubio/ejercicios-react-alternancia	COMPLETADO
Despliegue Vercel	ejercicios-react-alternancia.vercel.app	COMPLETADO
Despliegue InfinityFree	ejercicios-react-julian.gt.tc	COMPLETADO
Memoria de Proyecto	Fichero <code>DOCUMENTACION.md</code> y <code>DOCUMENTACION.html</code>	COMPLETADO

Resultados de Aprendizaje Cubiertos (RA)

1. RA3 — Desarrollo de código en React utilizando componentes funcionales

Se ha diseñado la aplicación entera utilizando el estándar moderno de React 18, abstrayéndose del modelo clásico de clases y utilizando componentes funcionales limpios, reactivos y fácilmente legibles.

- **Hooks Utilizados:** `useState` para control reactivo de estados (edición inline, galería, filtros, tema oscuro) y `useEffect` para la persistencia local de preferencias en el cliente.
- **Componentización Modular:** Creación de módulos reutilizables altamente cohesionados e independientes (como las tarjetas individuales de posts y el conmutador de tema).

2. RA5 — Programación basada en eventos y control de flujos de datos

Control de flujos del flujo de trabajo del usuario capturando eventos tanto interactivos (teclado, ratón) como de foco y ciclo de vida de componentes.

- **Eventos de Entrada:** Manejo inteligente de `onChange` para re-evaluar la validez de los formularios carácter a carácter solo cuando sea estrictamente necesario.
- **Eventos de Foco:** Uso de `onBlur` para detectar la interacción del usuario con un campo concreto y no penalizar visualmente con alertas de error antes de que el usuario termine de rellenar los datos.
- **Interceptación de Envíos:** Uso de `preventDefault()` para evitar el comportamiento predeterminado del navegador, validando el contenido completo en cliente antes de procesar el envío de datos.

3. RA6 — Enrutamiento y Single Page Applications (SPA)

Desarrollo estructurado utilizando el estándar de la industria **React Router DOM v6** para ofrecer una experiencia fluida e interactiva de aplicación de página única.

- **Routing Limpio:** Enrutamiento sin recargas mediante componentes declarativos como `BrowserRouter` y `Routes`.
- **Navegación Activa:** Navegación optimizada mediante componentes `<NavLink>` que inyectan clases CSS específicas dinámicamente según la ruta seleccionada por el usuario de forma totalmente imperativa.
- **Ruta de Escape 404:** Implementación de una página de desvío 404 adaptada e integrada en el layout global para capturar errores de enrutamiento aleatorios.

4. RA7 — Despliegue, optimización y publicación de aplicaciones web

Ejecución y puesta en producción en múltiples entornos remotos utilizando herramientas avanzadas de compilación y empaquetado.

- **Optimización con Vite:** Ejecución de `npm run build` para modularizar, minificar y optimizar el peso del bundle final de CSS y JS en la carpeta `dist/`.
- **CI/CD (Vercel):** Despliegue automático e instantáneo conectado con GitHub, garantizando que cada commit a la rama principal se integre en producción de forma transparente en menos de un minuto.
- **Hosting Tradicional (InfinityFree):** Transferencia FTP clásica de los ficheros del build a la carpeta pública del servidor remoto `htdocs/`.

Justificación Técnica de los Ejercicios Realizados

Ejercicio 1 — Navegación de una SPA con React Router DOM

Se ha creado un layout con un componente barra de navegación `Navbar`. Emplea un menú de tipo hamburguesa reactivo en dispositivos móviles que conmuta su visibilidad gracias a un boolean en `useState`. La navegación se realiza de manera imperativa y con una óptima experiencia fluida sin parpadeos.

Ejercicio 2 — Formulario con Validación en Tiempo Real

El componente `ContactForm` realiza una validación exhaustiva de los tres campos de entrada de datos:

- Comprobación de obligatoriedad en todos los campos.
- Validación sintáctica del correo electrónico mediante expresión regular (regex).
- Validación de longitud de caracteres mínima para el cuadro de mensaje, forzando al usuario a redactar al menos 20 caracteres para evitar spam.

El feedback visual cambia en vivo el color de borde de cada caja (verde o rojo), mostrando iconos claros indicando el estado actual de cada campo.

Ejercicio 3 — Galería de Imágenes Interactiva con Animaciones

El componente `Gallery` implementa un visor modularizado que renderiza las miniaturas a través de un mapeado de un array de objetos pasados como propiedades (props). Al pulsar en una miniatura, se actualiza el índice activo en el visor de manera limpia. Además, se ha integrado la librería **Framer Motion** para añadir un sutil fundido cruzado (*cross-fade*) en las transiciones de imagen, creando un efecto premium espectacular.

Ejercicio 4 — Sistema de Publicaciones Dinámicas (Blog CRUD)

Se ha implementado un CRUD local para gestionar publicaciones de blog directamente en el DOM mediante manipulación de estados inmutables:

- **Create (Crear):** Formulario para inyectar nuevos posts al inicio del listado.
- **Read (Leer):** Visualización interactiva mediante tarjetas modularizadas `PostCard`.
- **Update (Editar):** Activación de campos input inline dentro de la propia tarjeta para editar título y cuerpo sin interrumpir el flujo.
- **Delete (Borrar):** Función de borrado con confirmación integrada.
- **Ordenación Especial:** Posibilidad de destacar publicaciones (★). Los artículos destacados son ordenados en primera posición de forma algorítmica y cuentan con una estética dorada premium distintiva.

Ejercicio 5 — Modo Oscuro / Claro Alternable

Control centralizado a nivel de raíz (`App.jsx`) que utiliza el hook `useState` para conmutar dinámicamente un atributo `data-theme` en el nodo principal `<html>` . En el archivo CSS general `index.css` se mapean todas las variables de color mediante el formato HSL, facilitando una transición fluida al cambiar de tema sin parpadeos de luz en la pantalla. La preferencia del usuario es guardada en la base de datos local `localStorage` .

Estructura del Código Subido

```
src/
├─ main.jsx          # Punto de entrada de la aplicación
├─ App.jsx           # Router raíz, gestión de Modo Oscuro e inyección de layouts
├─ App.css           # Estilos globales de rejilla y layout
├─ index.css         # Definición de variables HSL de tema claro y oscuro
|
├─ components/       # Componentes auxiliares
|   └─ Navbar.jsx     # Barra de navegación interactiva y responsive
|   └─ Navbar.css
|   └─ ThemeToggle.jsx # Botón flotante animado para el tema de luz/oscuridad
|   └─ ThemeToggle.css
|   └─ PostCard.jsx   # Tarjeta del Blog con estado CRUD de lectura/edición
|   └─ PostCard.css
|
├─ pages/            # Vistas principales del enrutador
|   └─ Inicio.jsx     # Landing con Hero premium, stack técnico e info del autor
|   └─ Inicio.css
|   └─ Servicios.jsx  # Contenedor de Galería e interactividad del Blog
|   └─ Servicios.css
|   └─ Contacto.jsx   # Contenedor del Formulario e información del centro
|   └─ Contacto.css
|   └─ NotFound.jsx   # Vista 404 personalizada ante rutas inexistentes
|   └─ NotFound.css
|
└─ features/         # Componentes lógicos de cada ejercicio
    └─ ContactForm.jsx # Ejercicio 2 (Validación de formulario)
    └─ ContactForm.css
    └─ Gallery.jsx     # Ejercicio 3 (Galería de imágenes y animaciones)
    └─ Gallery.css
    └─ Blog.jsx        # Ejercicio 4 (CRUD completo de posts)
    └─ Blog.css
```

Proyecto desarrollado para la materia de Desarrollo Web en Entorno Cliente (Ejercicios en Alternancia).